ЗВІТ

про виконання лабораторної роботи №5

«Управління станом застосунку з використанням StatefulWidget»

з дисципліни

«Програмування мобільних додатків»

Студента групи КН-2327Б

Багрія-Белза Андрія

Завдання: 1. Продовжити роботу над проєктом, розпочатим у Лабораторній роботі №4.

2. Екран повинен бути реалізований як StatefulWidget.

3. Повинен бути створений окремий клас стану (_ScreenNameState).

4. Всі змінні стану повинні зберігатися всередині класу State.

5. Оновлення інтерфейсу повинно виконуватись виключно через setState().

6. Обов’язкові змінні стану (мінімум): bool — для Switch / Checkbox / обраного стану; int — лічильник дії; String — значення з TextField.

7. Обов’язкові інтерактивні елементи:

- Іконка «обране» (змінює колір при натисканні, реалізується через bool);
- Кнопка дії (збільшує лічильник, відображає поточне значення на екрані);
- Checkbox або Switch (змінює відображення: показ/приховування тексту, зміна стилю, поява бейджа, зміна кольору);
- TextField (зчитує введений текст, відображає результат нижче, використовує onChanged).

1 Написання коду-розв’язку завдання:

```
// === lib/screens/lab5_screen.dart ===
class _Lab5ScreenState extends State<Lab5Screen> {
  // 1. bool — стан іконки "обране"
  bool _isFavorite = false;

  // 2. bool — стан перемикача терміновості (Switch)
  bool _isUrgent = false;

  // 3. int — лічильник дій користувача
  int _executionCount = 0;

  // 4. String — значення, введене у TextField
  String _note = '';

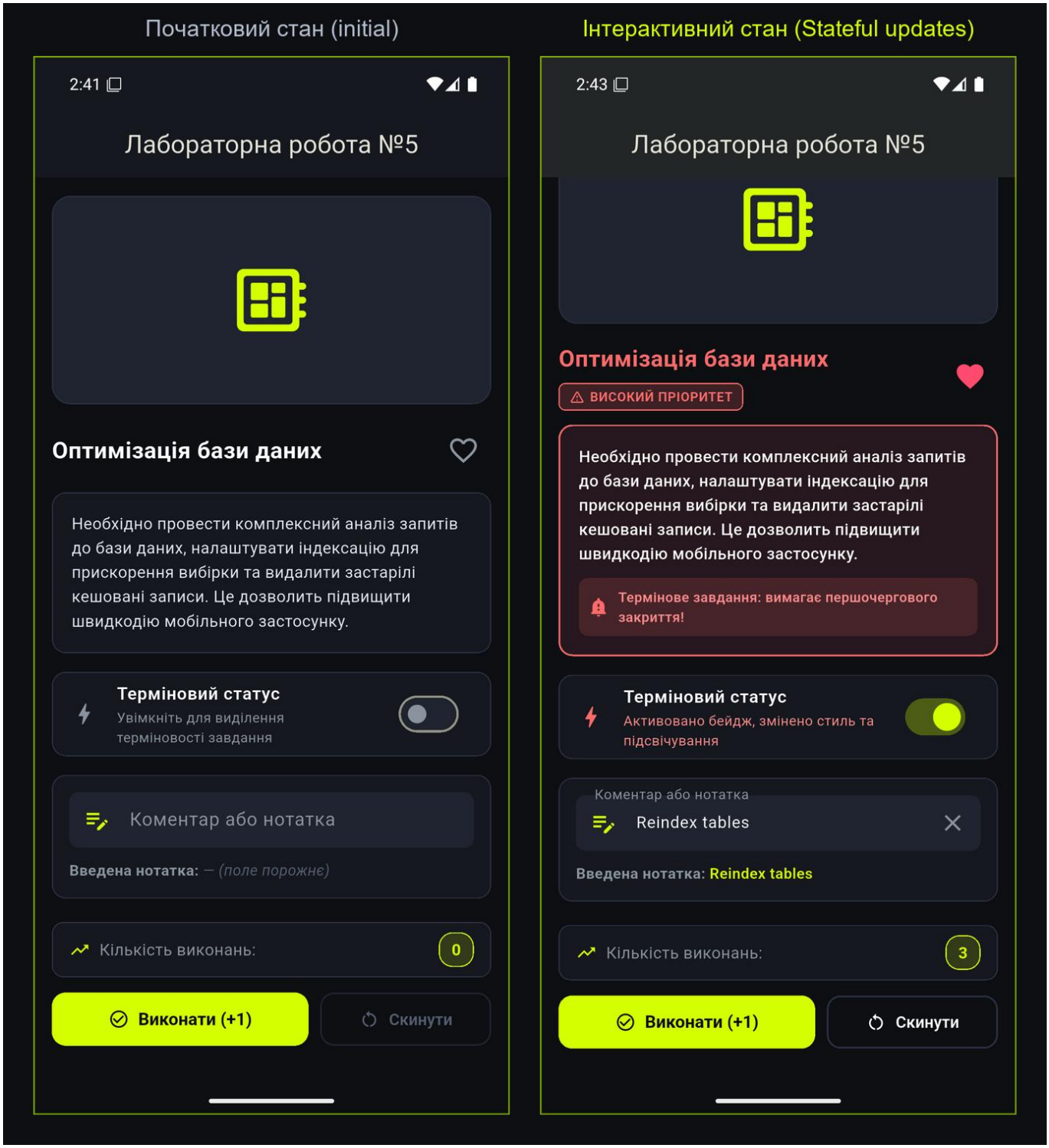
  final TextEditingController _textController = TextEditingController();

  @override
  void dispose() {
    _textController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
  // ... (пропуск неістотних рядків)
              // 1. Заголовок + іконка "обране" (bool _isFavorite)
              TaskHeader(
                title: 'Оптимізація бази даних',
                isFavorite: _isFavorite,
                isUrgent: _isUrgent,
                onFavoriteToggle: () {
                  setState(() {
                    _isFavorite = !_isFavorite;
```

```
                  });
                  _showMessage(
                    _isFavorite ? 'Додано до обраного ❤️‍🩹' : 'Видалено з обраного 🤍',
                  );
                },
              ),
  // ... (пропуск неістотних рядків)
              // 5. Кнопка дії (int _executionCount) — збільшує лічильник
              ActionButtonsRow(
                executionCount: _executionCount,
                onExecute: () {
                  setState(() {
                    _executionCount++;
                  });
                  _showMessage('Дію зафіксовано! Всього: $_executionCount');
                },
  // ... (пропуск неістотних рядків)
  }
}
```

2 Результат виконання коду:



1. Що таке стан (state) у Flutter-застосунку? Стан (State) — це дані або сукупність інформації, яка може динамічно змінюватися під час життєвого циклу застосунку (внаслідок взаємодії користувача, відповідей сервера чи таймерів) та безпосередньо впливає на відображення графічного інтерфейсу (UI). У реактивній моделі Flutter інтерфейс є функцією від стану: UI = f(State).

2. Яке призначення методу createState()? Метод createState() є обов'язковим перевизначеним методом класу StatefulWidget, який фреймворк викликає для інстанціювання та прив'язки відповідного екземпляра окремого класу стану (State<T>). Саме цей об'єкт State зберігає довготривалий змінний стан віджета в дереві елементів (Element Tree).

3. Для чого використовується метод setState()? Метод setState() слугує для повідомлення фреймворку Flutter про те, що внутрішній стан об'єкта State зазнав змін. Він приймає синхронний зворотний виклик (callback), у якому оновлюються поля стану, та сигналізує рушію рендерингу про необхідність перемалювання інтерфейсу.
4. Що відбувається після виклику setState()? Після виклику setState() відповідний Element віджета позначається як «брудний» (dirty) і планується до перебудови у черговому кадрі рендерингу (SchedulerBinding). Flutter викликає метод build(), зіставляє нове віджетне дерево з попереднім і точково оновлює змінені вузли в дереві рендерингу (RenderObject Tree).
5. Чому не рекомендується виконувати довгі обчислення всередині setState()? Метод setState() та передана до нього функція виконуються синхронно в головному потоці інтерфейсу (UI thread). Виконання важких обчислень чи синхронного введення/виведення призводить до блокування потоку, пропуску кадрів (jank / frame drops), сповільнення анімацій та зниження плавності взаємодії з користувачем.
6. Які методи життєвого циклу має StatefulWidget? Основні етапи та методи життєвого циклу: createState() (створення стану), initState() (одноразова ініціалізація під час монтування), didChangeDependencies() (викликається при зміні InheritedWidget), build() (побудова UI), didUpdateWidget() (оновлення конфігурації батьком), setState() (планування повторного build), deactivate() (вилучення з дерева) та dispose() (остаточне звільнення контролерів і ресурсів перед знищенням).
7. Як реалізувати оновлення тексту під час введення даних у TextField? Це реалізується шляхом передачі функції зворотного виклику в параметр onChanged віджета TextField (наприклад: onChanged: (val) { setState(() { myText = val; }); }). Під час введення символу TextField передає актуальний рядок, значення зберігається у змінній стану, а setState() запускає перебудову залежного віджету Text(myText).
8. Як зробити кнопку неактивною залежно від стану змінної? У кнопках Flutter (ElevatedButton, OutlinedButton, IconButton) параметр onPressed приймає значення типу VoidCallback?. Якщо передати null замість функції за певної умови (наприклад: onPressed: isEnabled ? () => doAction() : null), фреймворк автоматично переводить кнопку в неактивний стан, змінює її кольорову гаму на приглушену та блокує обробку натискань.
9. Які типи даних найчастіше використовуються для збереження стану? Найчастіше використовуються: bool (для перемикачів Switch, Checkbox, прапорців видимості чи обраного стану), int / double (для лічильників, числових значень, слайдерів), String (для збереження тексту з полів введення), List / Set / Map (для динамічних колекцій елементів) та призначені для користувача класи моделей і enum (для статусів процесів).
10. Що станеться, якщо змінити змінну стану без виклику setState()? Змінна в пам'яті буде модифікована, проте фреймворк Flutter не отримає сповіщення про зміну даних. Екран не запланує перебудову, і користувацький інтерфейс залишиться зі

застарілими відомостями доти, доки віджет випадково не перебудується з інших причин (наприклад, через глобальне оновлення батьківського вузла).

11. У яких випадках доцільно використовувати StatefulWidget? StatefulWidget доцільно використовувати, коли компонент інтерфейсу володіє власним динамічним станом, що змінюється внаслідок взаємодії користувача (форми введення, прапорці, анімації, лічильники), коли необхідно керувати життєвим циклом контролерів (TextEditingController, AnimationController, ScrollController) або відображати асинхронні зміни даних у межах одного екрана.

Висновок: На цій лабораторній роботі, я опанував механізми управління ефемерним станом у Flutter за допомогою StatefulWidget та класу State, реалізував динамічне оновлення інтерфейсу через виклики setState(), використав обов'язкові змінні стану різних типів (bool для обраного та перемикача терміновості, int для лічильника дій, String для зчитування тексту через onChanged у TextField), забезпечив зміну стилів, кольорів, показ бейджа та додаткового тексту відповідно до змін стану, а також перевірив стабільність роботи застосунку за допомогою автоматизованих тестів та на фізичному емуляторі Android.